RANCAK POS

Backend API Reference

Panduan Pengembangan Frontend Android

Platform POS Multi-Tenant dengan Fitur Offline-First

Versi Dokumen: 2.0 | Diperbarui: April 2026

Rancak Platform

Backend: Rust + Axum 0.7 + SQLx 0.8 + PostgreSQL
Android: Kotlin + Jetpack Compose + Room + Retrofit + WorkManager
Fitur Utama: Offline-First POS, QRIS Xendit, ESC/POS Bluetooth
Arsitektur: Multi-tenant, JWT Auth, Role-Based Access (Owner/Admin/Staff)

Daftar Isi

Daftar Isi 2
1. Arsitektur Backend 5
1.1 Stack Teknologi 5
1.2 Model Multi-Tenant 5
1.3 Hierarki Role 5
1.4 Format Base URL 5
2. Autentikasi 6
2.1 Login 6
2.2 Google Login (SSO) 6
2.3 Refresh Token 6
2.4 Logout 6
2.5 Header Auth Wajib 7
2.6 Endpoint Auth Lainnya 7
3. Konvensi API 8
3.1 Format Response 8
3.2 HTTP Status Codes 8
3.3 Idempotency Key 8
3.4 Pagination 8
4. Offline-First Architecture 10
4.1 Prinsip Utama 10
4.2 Endpoint Offline Sync 10
4.3 Alur Catalog Sync (Full) 10
4.4 Alur Delta Sync (Incremental) 10
4.5 Alur Sync Status (Ringan) 11
4.6 Alur Batch Sales (Sync Transaksi Offline) 11
5. Produk 13
5.1 Endpoint Produk 13
5.2 Format Response Produk 13
5.3 Catatan Penting 13
6. Kategori 15
7. Penjualan (Sales) 16
7.1 Endpoint Sales 16
7.2 Membuat Transaksi (POST /sales) 16
7.3 Alur Pembayaran QRIS 17
7.4 Format Respons Transaksi 17

8. Shift Kasir 19
9. Meja (Tables) 20
10. Kitchen Display System (KDS) 21
11. Surcharge, Pajak, Voucher & Diskon 22
11.1 Surcharge 22
11.2 Konfigurasi Pajak 22
11.3 Voucher 22
11.4 Discount Rules (Otomatis) 22
12. Kas Masuk & Pengeluaran 24
12.1 Kas Masuk (Cash In) 24
12.2 Pengeluaran (Expenses) 24
13. Bundle / Paket 25
14. Modifier (Catatan Cepat) 26
15. Perangkat & Konfigurasi Printer 27
15.1 Konfigurasi Printer 27
15.2 App Config (Key-Value) 27
15.3 Buka Laci Kas 27
15.4 ESC/POS Printing 27
16. Order Board 29
17. Laporan (Reports) 30
18. Android Tech Stack yang Direkomendasikan 31
18.1 Core Stack 31
18.2 Struktur Folder yang Disarankan 31
19. Panduan Implementasi Offline-First 33
19.1 Room DB Entities 33
19.2 SyncWorker (Background Upload) 33
19.3 Membuat Sale Offline 34
20. Checklist Header HTTP Android 35
21. Alur Kerja Lengkap Aplikasi POS Android 36
21.1 Startup Flow 36
21.2 Alur Transaksi Online 36
21.3 Alur Transaksi Offline 36
21.4 Token Refresh Flow 36
22. Nilai Enum & Konstanta 38
22.1 Tipe Data Enum di Backend 38
22.2 Format Mata Uang 38

23. Checklist Implementasi Android 39
23.1 Phase 1: Core (MVP).....39
23.2 Phase 2: Offline & Advanced 39
23.3 Phase 3: Manajemen (Admin) 39

1. Arsitektur Backend

1.1 Stack Teknologi

Rancak Backend dibangun dengan:

- **Rust + Axum 0.7** — framework web async, sangat cepat dan type-safe
- **SQLx 0.8 + PostgreSQL** — query async compile-time verified
- **JWT RS256** — access token + refresh token dengan rotation
- **Cloudflare Images** — CDN untuk gambar produk
- **Xendit QRIS** — integrasi pembayaran QR Code

1.2 Model Multi-Tenant

Setiap request ke endpoint tenant-scoped melewati dua layer autentikasi:

- **Layer 1 — JWT Valid:** middleware require_auth memverifikasi token
- **Layer 2 — Tenant Membership:** middleware require_tenant memverifikasi user adalah anggota tenant dan subscription aktif

Semua data di database selalu difilter dengan tenant_id. Android tidak bisa mengakses data tenant lain meskipun token valid.

1.3 Hierarki Role

Role	Level	Akses
`Staff`	1 (terendah)	Operasi kasir: buat sale, buka/tutup shift, KDS, cash-in, expense
`Admin`	2	Semua Staff + manajemen produk, kategori, meja, laporan, konfigurasi printer
`Owner`	3 (tertinggi)	Semua Admin + pengaturan tenant, surcharge, pajak, voucher, discount rules, laporan keuangan

1.4 Format Base URL

Semua endpoint tenant-scoped menggunakan pola:

```
Base URL  : https://api.rancak.id
Auth APIs : /auth/login, /auth/refresh, dll
Protected : /auth/me, /auth/sessions, dll
Tenant    : /tenants/:tenant_uuid/<resource>

Contoh:
  GET  https://api.rancak.id/tenants/abc-123/products
  POST https://api.rancak.id/tenants/abc-123/sales
  GET  https://api.rancak.id/tenants/abc-123/sync/catalog
```

2. Autentikasi

2.1 Login

Android mengirim email dan password. Server mengembalikan access token dan refresh token.

```
POST /auth/login

Request:
{
  "email":    "kasir@restoran.com",
  "password": "rahasia123"
}

Response 200:
{
  "status": "ok",
  "data": {
    "access_token":       "eyJhbGci...",
    "token_type":         "Bearer",
    "expires_in":         3600,
    "refresh_token":      "uuid-refresh-...",
    "refresh_expires_at": "2026-05-13T10:00:00Z",
    "user": {
      "uuid":    "user-uuid-...",
      "name":    "Budi Kasir",
      "email":   "kasir@restoran.com",
      "tenants": [
        { "uuid": "tenant-uuid-...", "name": "Warung Pak Budi" }
      ]
    }
  }
}
```

2.2 Google Login (SSO)

```
POST /auth/google
{
  "id_token": "<Google ID Token dari Google Sign-In SDK>"
}
```

2.3 Refresh Token

Access token berlaku 1 jam. Gunakan refresh token untuk mendapatkan token baru tanpa login ulang. Setiap refresh menghasilkan pasangan token baru (rotation).

```
POST /auth/refresh
{
  "refresh_token": "uuid-refresh-..."
}
→ Response sama dengan login: access_token + refresh_token baru
```

2.4 Logout

```
POST /auth/logout
{
  "refresh_token": "uuid-refresh-..."  // invalidate session ini
}
```

2.5 Header Auth Wajib

Setiap request ke endpoint protected harus menyertakan:

```
Authorization: Bearer <access_token>
X-API-Key:     <api_key>          // Opsional - wajib jika backend dikonfigurasi
Content-Type:  application/json
```

2.6 Endpoint Auth Lainnya

Method	Endpoint	Role Min	Keterangan
GET	/auth/me	Any	Profil user + daftar tenant yang diikuti
POST	/auth/change-password	Any	Ganti password: {current_password, new_password}
POST	/auth/forgot-password	Public	Kirim email reset: {email}
POST	/auth/reset-password	Public	Reset dengan token email: {token, new_password}
GET	/auth/sessions	Any	List semua sesi login aktif user ini
DELETE	/auth/sessions/:session_id	Any	Paksa logout sesi lain (revoke refresh token)

3. Konvensi API

3.1 Format Response

Semua response menggunakan envelope yang konsisten:

```
// Sukses
{
  "status":  "ok",
  "message": "Sale created",  // opsional
  "data":    { ... }          // payload utama
}

// Error
{
  "status":  "error",
  "message": "Produk tidak ditemukan",
  "code":    404
}
```

3.2 HTTP Status Codes

HTTP	Status	Kapan
200	ok	Request berhasil
400	bad_request	Input tidak valid (kolom wajib kosong, format salah)
401	unauthorized	Token tidak ada atau expired → lakukan refresh
403	forbidden	Role tidak cukup untuk operasi ini
404	not_found	Resource tidak ditemukan
409	conflict	Idempotency key sudah diproses (normal, bukan error)
422	unprocessable	Validasi bisnis gagal (stok habis, dll)
500	internal_error	Error server → tampilkan pesan generik ke user

3.3 Idempotency Key

Untuk operasi penting (POST sale, batch sync), Android wajib mengirim header idempotency key. Ini mencegah duplikat jika request dikirim ulang karena timeout.

```
Header:  X-Idempotency-Key: <UUID v4 unik per transaksi>

Contoh:  X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

// Jika key sama dikirim ulang → server kembalikan response yang sama
// TANPA membuat transaksi ganda. Ini adalah perilaku yang benar.
```

Aturan penggunaan: generate UUID sekali di Android saat user konfirmasi pembayaran. Simpan di Room DB bersama data transaksi. Gunakan key yang sama untuk semua retry.

3.4 Pagination

List endpoint mendukung paginasi dengan query params:

```
?limit=50&page=1

// limit: jumlah item per halaman (default bervariasi per endpoint)
// page:  nomor halaman mulai dari 1
```

4. Offline-First Architecture

4.1 Prinsip Utama

Aplikasi POS Android Rancak dirancang offline-first: transaksi bisa dibuat dan disimpan lokal bahkan tanpa internet. Data akan otomatis disinkronkan ke server saat koneksi tersedia.

- **Room DB** sebagai Single Source of Truth untuk semua data
- **WorkManager** untuk background sync dengan network constraint
- **ConnectivityManager** untuk deteksi real-time status internet
- **Idempotency Key** untuk mencegah duplikat saat retry

4.2 Endpoint Offline Sync

Method	Endpoint	Role Min	Keterangan
GET	/tenants/:id/sync/catalog	Staff	Bundle semua data offline: produk, kategori, surcharge, pajak, diskon, meja, shift
GET	/tenants/:id/sync/status	Staff	Cek timestamp server — apakah perlu sync (tanpa unduh semua data)
POST	/tenants/:id/sales/batch	Staff	Kirim array transaksi offline sekaligus dengan idempotency key per item

4.3 Alur Catalog Sync (Full)

Panggil saat app pertama kali dipasang atau user force refresh:

```
GET /tenants/:id/sync/catalog

Response:
{
  "data": {
    "server_time":     "2026-04-13T10:00:00Z",  // SIMPAN ini di SharedPrefs
    "is_delta":        false,
    "categories":      [...],
    "products":        [{ "uuid", "sku", "barcode", "name", "category_uuid",
                          "price", "stock", "unit", "is_active",
                          "updated_at" }],
    "surcharges":      [...],
    "tax_configs":     [...],
    "discount_rules": [...],
    "tables":          [...],
    "current_shift":   { "uuid", "opening_cash", "opened_at" } | null
  }
}
```

4.4 Alur Delta Sync (Incremental)

Panggil saat app dibuka ulang untuk sinkronisasi perubahan saja:

```
GET /tenants/:id/sync/catalog?updated_after=2026-04-13T10:00:00Z
```

```
// Kirim server_time dari sync sebelumnya sebagai updated_after
// Server hanya kembalikan produk/kategori yang berubah sejak waktu itu
// Response sama persis, tapi list lebih pendek (hanya yang berubah)

// Android: merge ke Room DB (upsert berdasarkan UUID)
```

4.5 Alur Sync Status (Ringan)

Panggil saat app foreground untuk cek apakah ada data baru tanpa mengunduh semua:

```
GET /tenants/:id/sync/status

Response:
{
  "data": {
    "server_time":           "2026-04-13T10:05:00Z",
    "products_updated_at":   "2026-04-13T09:30:00Z",
    "categories_updated_at": "2026-04-13T08:00:00Z",
    "has_open_shift":        true
  }
}

// Android: bandingkan products_updated_at dengan timestamp lokal
// Jika server lebih baru → trigger delta sync
```

4.6 Alur Batch Sales (Sync Transaksi Offline)

Kirim semua transaksi yang dibuat offline ke server dalam satu request:

```
POST /tenants/:id/sales/batch

Request:
{
  "sales": [
    {
      "idempotency_key":  "550e8400-...",  // UUID dari saat transaksi dibuat
offline
      "device_created_at": "2026-04-13T09:15:00Z",  // waktu di device saat
transaksi
      "device_id":         "android-device-id-xyz",
      "items":            [{ "product_uuid": "...", "qty": 2 }],
      "payment_method":   "cash",
      "paid_amount":      50000
    },
    { "idempotency_key": "...", ... }
  ]
}

Response:
{
  "data": {
    "total":     2,
    "created":   1,
    "duplicate": 1,
    "errors":    0,
    "results": [
      { "idempotency_key": "...", "status": "created",   "invoice_no": "INV-001",
"sale_uuid": "..." },
```

```
      { "idempotency_key": "...", "status": "duplicate", "invoice_no": "INV-000",
"sale_uuid": "..." }
    ]
  }
}
```

Status tiap item bisa: created (berhasil baru), duplicate (sudah pernah diproses), error (gagal). Android update Room DB sesuai hasil.

5. Produk

5.1 Endpoint Produk

Method	Endpoint	Role Min	Keterangan
GET	/products	Staff	List produk. Query: q, category_id, updated_after, limit, page
GET	/products/:uuid	Staff	Detail satu produk lengkap dengan varian
GET	/products/barcode/:barcode	Staff	Cari produk via barcode scanner
GET	/products/favorites	Staff	Produk favorit/sering dipesan (untuk shortcut di layar kasir)
GET	/products/86	Staff	List produk yang di-86 (habis sementara hari ini)
POST	/products/86	Staff	Mark produk habis: {"product_uuid":"..."}
DELETE	/products/86/:uuid	Staff	Unmark produk 86
GET	/products/:id/batches	Admin	List batch produk (FIFO/expiry tracking)
POST	/products/:id/batches	Admin	Tambah batch baru
POST	/products/:id/stock-adjustment	Admin	Koreksi stok manual
GET	/products/:id/variant-groups	Staff	List grup varian (Ukuran, Suhu, dll)
POST	/products/:id/variant-groups	Admin	Buat grup varian baru
GET	/products/:id/variant-groups/:gid/variants	Staff	List varian dalam satu grup
POST	/products/:id/image	Admin	Upload gambar produk (multipart/form-data)

5.2 Format Response Produk

```
{
  "uuid":          "uuid-produk",
  "sku":           "SKU001",
  "barcode":       "8991234567890",
  "name":          "Nasi Goreng Spesial",
  "description":   "...",
  "category":      { "uuid": "...", "name": "Makanan Berat" },
  "price":         35000,      // integer Rupiah
  "stock":         "99.50",    // Decimal (bisa pecahan untuk bahan baku)
  "unit":          "porsi",
  "image_url":     "https://imagedelivery.net/...",
  "is_active":     true,
  "has_expiry":    false,
  "updated_at":    "2026-04-10T08:00:00Z"  // untuk delta sync
}
```

5.3 Catatan Penting

- **Delta sync:** gunakan ?updated_after= untuk sync incremental. Room DB: upsert berdasarkan UUID
- **Barcode scan:** gunakan ML Kit di Android, kirim ke /barcode/:barcode
- **86 reset:** otomatis direset tiap hari oleh background job server
- **Varian:** produk bisa punya beberapa grup varian (mis. Ukuran: S/M/L, Suhu: panas/dingin). Harga varian adalah harga dasar + price_adjustment

6. Kategori

Method	Endpoint	Role Min	Keterangan
GET	/categories	Staff	List semua kategori tenant
GET	/categories/:id	Staff	Detail satu kategori
POST	/categories	Admin	Buat kategori: {name, description}
PATCH	/categories/:id	Admin	Update: {name?, description?}
DELETE	/categories/:id	Admin	Hapus kategori (soft delete)

Response kategori: {uuid, name, description, created_at}. Setelah migration 0030, kategori juga memiliki updated_at untuk delta sync via catalog bundle.

7. Penjualan (Sales)

7.1 Endpoint Sales

Method	Endpoint	Role Min	Keterangan
GET	/sales	Staff	List penjualan. Query: date_from, date_to, status, mine, limit, page
POST	/sales	Staff	Buat transaksi baru (WAJIB X-Idempotency-Key)
GET	/sales/:uuid	Staff	Detail transaksi lengkap
POST	/sales/:uuid/pay	Staff	Bayar hold order yang sudah ada
POST	/sales/:uuid/cancel	Staff	Batalkan hold order
POST	/sales/:uuid/items	Staff	Tambah item ke hold order
DELETE	/sales/:uuid/items/:item_id	Staff	Hapus item dari hold order
POST	/sales/:uuid/void	Admin	Void/batalkan transaksi yang sudah paid
POST	/sales/:uuid/refund	Admin	Refund sebagian atau penuh
POST	/sales/:uuid/move-table	Staff	Pindah meja: {table_uuid}
POST	/sales/:uuid/merge	Admin	Merge dua sale ke satu
POST	/sales/:uuid/serve	Staff	Tandai pesanan sudah diantar (status served)
POST	/sales/:uuid/split	Staff	Split bill: pisahkan item ke transaksi baru
GET	/sales/:uuid/receipt	Staff	Data receipt lengkap (untuk render di app)
GET	/sales/:uuid/receipt/escpos	Staff	Raw ESC/POS bytes untuk printer thermal 80mm
GET	/sales/:uuid/receipt/kitchen	Staff	ESC/POS nota dapur
GET	/sales/:uuid/receipt/combined	Staff	ESC/POS kasir+dapur digabung
GET	/sales/:uuid/receipt/queue	Staff	ESC/POS nomor antrian
POST	/sales/:uuid/reprint	Staff	Tandai sudah reprint
POST	/sales/batch	Staff	Batch submit transaksi offline (lihat Offline Sync)

7.2 Membuat Transaksi (POST /sales)

```
POST /tenants/:id/sales
X-Idempotency-Key: <uuid-unik>

{
  // === WAJIB ===
  "items": [
    {
      "product_uuid":  "uuid-produk",     // atau bundle_uuid untuk paket
```

```
      "qty":           2,
      "variant_uuid":  "uuid-varian",    // opsional
      "price":         35000,            // opsional: override harga
      "discount":      5000,             // opsional: diskon per item (Rp)
      "discount_pct":  10,               // opsional: diskon per item (%)
      "override_reason":"Harga khusus",  // opsional: alasan override
      "note":          "Tidak pedas",    // catatan dari modifier chip
      "addons": [{ "name":"Extra kerupuk","price":1000,"qty":1 }]
    }
  ],

  // === PEMBAYARAN ===
  // Pilih salah satu: single payment ATAU split payment
  "payment_method": "cash",    // cash|card|qris|transfer|other
  "paid_amount":    70000,     // nominal yang dibayarkan pelanggan

  // Split payment (isi "payments" gantikan payment_method + paid_amount):
  "payments": [
    { "method": "cash", "amount": 50000 },
    { "method": "qris", "amount": 20000 }
  ],

  // === OPSIONAL ===
  "order_type":     "dine_in",    // dine_in|takeaway|delivery
  "customer_name":  "Pak Budi",
  "table_uuid":     "uuid-meja",
  "discount":       5000,         // diskon level transaksi (Rp)
  "tax":            3500,         // pajak (jika tidak ada tax_config)
  "tip":            2000,
  "admin_fee":      1000,
  "delivery_fee":   15000,
  "voucher_code":   "DISC10",
  "use_queue":      true,
  "note":           "Lantai 2",
  "hold":           false,        // true = simpan hold, bayar nanti

  // Offline-first (untuk sync dari device):
  "device_created_at": "2026-04-13T09:15:00Z",
  "device_id":         "android-device-xyz"
}
```

7.3 Alur Pembayaran QRIS

- **1.** Buat sale dengan payment_method: "qris"
- **2.** POST /sales/:uuid/qr-payment → server buat QR via Xendit, dapat qr_string
- **3.** Android render QR dari qr_string (gunakan ZXing atau QRose)
- **4.** Polling GET /sales/:uuid/qr-payment setiap 5 detik, atau pasang WebSocket jika tersedia
- **5.** Saat status "paid" → tampilkan sukses, cetak struk
- **PENTING:** QRIS tidak bisa digunakan saat offline. Android harus cek koneksi sebelum menampilkan opsi QRIS

7.4 Format Respons Transaksi

```
{
  "uuid":         "sale-uuid",
```

```
  "invoice_no":   "INV-2026/04/001",
  "order_type":   "dine_in",
  "queue_number": 5,
  "status":       "paid",    // held|paid|void|served|cancelled
  "subtotal":     70000,
  "discount":     5000,
  "surcharge":    2000,
  "voucher_discount": 3000,
  "auto_discount":    0,
  "tax":          7000,
  "delivery_fee": 0,
  "tip":          0,
  "admin_fee":    0,
  "total":        71000,
  "payment_method": "cash",
  "paid_amount":  80000,
  "change_amount":9000,
  "cashier_uuid": "...",
  "table_uuid":   "...",
  "items": [...],
  "payments": [...],
  "created_at":   "2026-04-13T09:15:00Z"
}
```

8. Shift Kasir

Kasir wajib membuka shift sebelum bisa membuat transaksi. Sistem tracking uang kas per shift.

Method	Endpoint	Role Min	Keterangan
POST	/shifts/open	Staff	Buka shift: {opening_cash}. Satu kasir hanya bisa punya 1 shift open
POST	/shifts/close	Staff	Tutup shift: {closing_cash, note}
GET	/shifts/current	Staff	Data shift yang sedang buka milik user ini
GET	/shifts/current/summary	Staff	Ringkasan penjualan shift aktif (total, per metode bayar)
GET	/shifts/:id/summary	Admin	Ringkasan shift yang sudah tutup
GET	/shifts/:id/cash-count	Staff	Cash count detail shift
POST	/shifts/:id/cash-count	Staff	Submit cash count manual

```
// Buka shift:
POST /shifts/open
{ "opening_cash": 100000 }  // uang awal di laci kas

// Tutup shift:
POST /shifts/close
{ "closing_cash": 450000, "note": "Shift pagi selesai" }

// Response summary:
{
  "uuid":         "shift-uuid",
  "opened_at":    "...",
  "closed_at":    null,
  "status":       "open",
  "opening_cash": 100000,
  "payment_summary": [
    { "method": "cash", "total": 350000 },
    { "method": "qris", "total": 100000 }
  ],
  "total_sales": 450000,
  "total_expenses": 50000
}
```

9. Meja (Tables)

Method	Endpoint	Role Min	Keterangan
GET	/tables	Staff	List semua meja dengan status terkini
GET	/tables/:id	Staff	Detail meja termasuk active_sale_uuid jika sedang digunakan
POST	/tables	Admin	Buat meja baru: {name, area, capacity, sort_order}
PATCH	/tables/:id	Admin	Update: {name?, area?, status?, is_active?, sort_order?}
DELETE	/tables/:id	Admin	Hapus meja (soft delete)

```
{
  "uuid":       "meja-uuid",
  "name":       "Meja 1",
  "area":       "Lantai 1",
  "capacity":   4,
  "status":     "available",  // available|occupied|reserved|maintenance
  "is_active": true,
  "sort_order":1,
  "active_sale_uuid": null   // UUID sale aktif jika occupied
}
```

Status meja berubah otomatis: occupied saat sale dibuat dengan table_uuid, available saat sale void/selesai/dibatalkan. Android bisa force-reset via PATCH /tables/:id status: "available".

10. Kitchen Display System (KDS)

KDS menampilkan pesanan yang masuk ke dapur. Android polling endpoint ini untuk memperbarui tampilan layar dapur.

Method	Endpoint	Role Min	Keterangan
GET	/kds	Staff	List KDS orders. Query: status (new|cooking|ready|done|active), date, limit, page
GET	/kds/:uuid	Staff	Detail satu KDS order
PATCH	/kds/:uuid	Staff	Update status order: {status: "cooking|ready|done"}
PATCH	/kds/:uuid/items/:item_uuid	Staff	Update status satu item: {status: "pending|cooking|ready|done"}

```
{
  "uuid":          "kds-uuid",
  "invoice_no":    "INV-001",
  "order_type":    "dine_in",
  "table_name":    "Meja 1",
  "queue_number": 5,
  "status":        "new",    // new|cooking|ready|done
  "items": [
    { "uuid":"...", "product_name":"Nasi Goreng", "qty":"1",
      "variant_name":"Spesial", "note":"Tidak pedas",
      "status":"pending"  // pending|cooking|ready|done
    }
  ],
  "created_at": "..."
}
```

Polling interval yang disarankan: 5 detik untuk KDS aktif, 30 detik saat app di background. Filter default active menampilkan semua pesanan new+cooking+ready.

11. Surcharge, Pajak, Voucher & Diskon

11.1 Surcharge

Surcharge ditambahkan otomatis berdasarkan order_type (dine_in, takeaway, delivery). Android tidak perlu menghitung manual — server yang menghitung.

Method	Endpoint	Role Min	Keterangan
GET	/surcharges	Staff	List semua surcharge (untuk informasi di layar kasir)
GET	/surcharges/:id	Admin	Detail surcharge
POST	/surcharges	Admin	Buat surcharge: {order_type, name, amount, is_percentage, max_amount}
PATCH	/surcharges/:id	Admin	Update surcharge

11.2 Konfigurasi Pajak

Method	Endpoint	Role Min	Keterangan
GET	/tax-configs	Staff	List semua tax config aktif
POST	/tax-configs	Owner	Buat: {name, rate, apply_to:"subtotal|after_discount"}
PATCH	/tax-configs/:id	Owner	Update

11.3 Voucher

Method	Endpoint	Role Min	Keterangan
GET	/vouchers	Admin	List voucher
GET	/vouchers/validate	Staff	Validasi kode: ?code=DISC10&subtotal=50000 → dapat discount_applied
POST	/vouchers	Admin	Buat voucher

Alur penggunaan di Android: saat kasir mengetik kode voucher, panggil GET /vouchers/validate untuk preview diskon, kemudian sertakan voucher_code di body POST /sales.

11.4 Discount Rules (Otomatis)

Method	Endpoint	Role Min	Keterangan
GET	/discount-rules	Staff	List semua aturan diskon otomatis
GET	/discount-rules/preview	Staff	Preview: ?total=50000 → berapa diskon yang berlaku
POST	/discount-rules	Admin	Buat rule: happy hour, min purchase, dll

Diskon rules dievaluasi otomatis oleh server saat POST /sales. Android bisa panggil /discount-rules/preview untuk menampilkan preview diskon ke kasir sebelum transaksi dikonfirmasi.

12. Kas Masuk & Pengeluaran

12.1 Kas Masuk (Cash In)

Method	Endpoint	Role Min	Keterangan
GET	/cash-ins	Staff	List kas masuk. Query: date_from, date_to, limit, page
POST	/cash-ins	Staff	Catat kas masuk: {amount, source, description, note}
GET	/cash-ins/:id	Staff	Detail satu cash in
DELETE	/cash-ins/:id	Staff	Hapus (hanya kasir yang membuat, hari yang sama)

12.2 Pengeluaran (Expenses)

Method	Endpoint	Role Min	Keterangan
GET	/expenses	Staff	List pengeluaran
POST	/expenses	Staff	Catat: {category_uuid?, amount, description, expense_date?, note?}
PATCH	/expenses/:id	Staff	Update
DELETE	/expenses/:id	Staff	Hapus
GET	/expenses/categories	Admin	List kategori pengeluaran
POST	/expenses/categories	Admin	Buat kategori

13. Bundle / Paket

Bundle adalah paket produk yang dijual sebagai satu item. Stok setiap komponen dipotong otomatis saat bundle dibeli.

Method	Endpoint	Role Min	Keterangan
GET	/bundles	Staff	List semua bundle aktif
GET	/bundles/:id	Staff	Detail bundle beserta komponennya
POST	/bundles	Admin	Buat bundle: {name, price, items:[{product_uuid, qty}]}
PATCH	/bundles/:id	Admin	Update (termasuk replace items)
DELETE	/bundles/:id	Admin	Hapus bundle (soft delete)

Di layar kasir, bundle tampil di tab tersendiri atau dapat difilter. Saat menambahkan ke cart, gunakan bundle_uuid (bukan product_uuid) di field items pada POST /sales.

14. Modifier (Catatan Cepat)

Modifier adalah preset catatan yang muncul sebagai chip di layar kasir. Tap chip menambahkan teks ke field note item. Tidak mempengaruhi harga.

Method	Endpoint	Role Min	Keterangan
GET	/modifiers	Staff	List semua modifier (global + per produk)
POST	/modifiers	Admin	Buat modifier global: {name, sort_order}
GET	/modifiers/product/:pid	Staff	Modifier untuk produk tertentu + global
POST	/modifiers/product/:pid	Admin	Buat modifier khusus produk
PATCH	/modifiers/:id	Admin	Update
DELETE	/modifiers/:id	Admin	Hapus

Di layar detail order, tampilkan modifier sebagai chip. Saat kasir memilih modifier, tambahkan nama modifier ke field note item.

15. Perangkat & Konfigurasi Printer

15.1 Konfigurasi Printer

Android menyimpan konfigurasi printer thermal (Bluetooth/Network/USB) ke server agar tersinkronisasi antar device.

Method	Endpoint	Role Min	Keterangan
GET	/device-config/printers	Staff	List printer milik device ini (filter by device_id header)
POST	/device-config/printers	Staff	Daftarkan printer: {device_id, printer_type, conn_type, bt_address/ip_address}
PATCH	/device-config/printers/:id	Staff	Update konfigurasi printer
DELETE	/device-config/printers/:id	Staff	Hapus printer
GET	/device-config/printers/all	Admin	Semua printer tenant (untuk admin)

```
// Daftarkan printer Bluetooth:
{
  "device_id":    "android-device-id",  // Settings.Secure.ANDROID_ID
  "device_name":  "Tablet Kasir 1",
  "printer_type": "receipt",   // receipt|kitchen|label
  "conn_type":    "bluetooth", // bluetooth|network|usb
  "bt_address":   "AA:BB:CC:DD:EE:FF",
  "paper_width":  80,          // 58 atau 80 (mm)
  "is_default":   true
}
```

15.2 App Config (Key-Value)

Method	Endpoint	Role Min	Keterangan
GET	/device-config/app	Staff	Semua config key tenant
POST	/device-config/app/:key	Admin	Set config: {value, description?}
DELETE	/device-config/app/:key	Admin	Reset ke default

15.3 Buka Laci Kas

Method	Endpoint	Role Min	Keterangan
GET	/devices/cash-drawer/open	Staff	Kembalikan raw ESC/POS bytes untuk buka laci kas

Android kirim bytes response ini langsung ke socket printer Bluetooth/IP. Panggil ini setelah pembayaran tunai berhasil.

15.4 ESC/POS Printing

Endpoint receipt menghasilkan raw ESC/POS bytes yang bisa langsung dikirim ke printer thermal:

```
GET /sales/:uuid/receipt/escpos    → 80mm receipt kasir
```

```
GET /sales/:uuid/receipt/kitchen   → nota dapur
GET /sales/:uuid/receipt/combined  → kasir+dapur dalam satu print
GET /sales/:uuid/receipt/queue     → nomor antrian

// Android: kirim bytes ke printer via Bluetooth atau Network socket
// BluetoothSocket.outputStream.write(responseBytes)
// Socket(ip, port).outputStream.write(responseBytes)
```

16. Order Board

Tampilan papan pesanan untuk kasir/masak single-person (seperti warung mie atau warteg). Menampilkan semua pesanan hari ini dalam urutan waktu.

Method	Endpoint	Role Min	Keterangan
GET	`/order-board`	Staff	List semua pesanan hari ini. Query: date, include_done, include_cancelled

```
GET /order-board?date=2026-04-13&include_done=true

Response: [
  {
    "uuid":         "sale-uuid",
    "invoice_no":   "INV-001",
    "queue_number":5,
    "order_type":   "takeaway",
    "status":       "paid",  // held|paid|served|void|cancelled
    "items": [
      { "product_name":"Indomie Goreng", "qty":"2", "note":"Tidak pakai bawang" }
    ],
    "created_at":   "..."
  }
]
```

17. Laporan (Reports)

Laporan hanya untuk role Admin dan Owner. Android bisa menampilkan laporan di tab terpisah untuk Admin.

Method	Endpoint	Role Min	Keterangan
GET	/reports/summary	Admin	Ringkasan penjualan: ?date_from&date_to → total, per metode bayar
GET	/reports/sales-by-cashier	Admin	Penjualan per kasir dalam rentang tanggal
GET	/reports/products	Admin	Produk terlaris: qty terjual, total pendapatan
GET	/reports/categories	Admin	Penjualan per kategori
GET	/reports/hourly	Admin	Peak hours: penjualan per jam
GET	/reports/payment-methods	Admin	Breakdown metode pembayaran
GET	/reports/void-refund	Admin	Laporan void dan refund
GET	/reports/pnl	Admin	Profit & Loss: pendapatan, pengeluaran, hpp
GET	/reports/stock-movement	Admin	Pergerakan stok
GET	/reports/daily	Admin	Laporan harian per shift
GET	/reports/shifts	Admin	List semua shift dengan ringkasan

18. Android Tech Stack yang Direkomendasikan

18.1 Core Stack

Komponen	Library	Kegunaan
Language	Kotlin 1.9+	Bahasa utama Android modern
UI	Jetpack Compose	UI deklaratif, mudah state management
Architecture	MVVM + Clean Architecture	ViewModel, UseCase, Repository
Local DB	Room 2.6+	SQLite ORM — offline-first, SSOT
Network	Retrofit 2 + OkHttp 4	HTTP client, interceptors untuk auth
DI	Hilt	Dependency injection otomatis
Coroutines	Kotlin Coroutines + Flow	Async, state streaming ke UI
Background Sync	WorkManager	Sync offline transactions ke server
JSON	Moshi atau Gson	Serialisasi request/response API
Navigation	Navigation Compose	Multi-screen navigation
Image	Coil 3	Load gambar produk dari Cloudflare CDN
QR Scan	ML Kit Barcode	Scan barcode produk, fully offline
QR Render	QRose / ZXing	Render QR code QRIS
Printer	ESC/POS Library (ESCPOS-ThermalPrinter)	Cetak struk via Bluetooth/Network
Google Auth	Google Sign-In SDK v2	Login dengan Google account
Connectivity	ConnectivityManager	Detect online/offline real-time

18.2 Struktur Folder yang Disarankan

```
app/
  data/
    local/          Room DAO + Entity
    remote/         Retrofit API interfaces + DTOs
    repository/     Implementasi Repository
  domain/
    model/          Business model classes
    usecase/        Use cases (CreateSaleUseCase, SyncCatalogUseCase, dll)
    repository/     Repository interfaces
  presentation/
    ui/             Composable screens
      pos/          Layar kasir utama
      cart/         Keranjang belanja
      payment/      Pilih metode bayar
      kds/          Kitchen display
      shifts/       Buka/tutup shift
      orderboard/   Order board
      reports/      Laporan (Admin)
    viewmodel/      ViewModels
    components/     Shared composables
```

```
  di/                Hilt modules
  worker/            WorkManager workers
    SyncWorker.kt    Upload offline transactions
    CatalogWorker.kt Download catalog updates
  utils/             Helpers (formatRupiah, ESCPOSHelper, dll)
```

19. Panduan Implementasi Offline-First

19.1 Room DB Entities

Buat entity untuk setiap data yang perlu offline:

```
@Entity(tableName = "products")
data class ProductEntity(
  @PrimaryKey val uuid: String,
  val sku: String,
  val name: String,
  val categoryUuid: String?,
  val price: Long,
  val stock: BigDecimal,
  val isActive: Boolean,
  val updatedAt: Long  // epoch millis — untuk delta sync
)

@Entity(tableName = "offline_sales")
data class OfflineSaleEntity(
  @PrimaryKey val idempotencyKey: String,  // UUID dibuat di device
  val saleJson: String,                    // JSON serialisasi CreateSaleInput
  val deviceCreatedAt: Long,
  val status: String  // PENDING | SYNCED | ERROR
)
```

19.2 SyncWorker (Background Upload)

```
class SyncWorker(ctx: Context, params: WorkerParameters) :
    CoroutineWorker(ctx, params) {

    override suspend fun doWork(): Result {
        val pendingSales = db.offlineSaleDao().getPendingSales()
        if (pendingSales.isEmpty()) return Result.success()

        val batchInput = BatchSalesInput(
            sales = pendingSales.map { it.toCreateSaleInput() }
        )

        return try {
            val response = api.batchSales(tenantUuid, batchInput)
            response.data.results.forEach { result ->
                when (result.status) {
                    "created", "duplicate" ->
                        db.offlineSaleDao().markSynced(result.idempotencyKey)
                    "error" ->
                        db.offlineSaleDao().markError(result.idempotencyKey)
                }
            }
            Result.success()
        } catch (e: Exception) {
            Result.retry()  // WorkManager akan retry dengan backoff
        }
    }
}

// Register dengan network constraint:
val syncRequest = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(Constraints(requiresNetwork = true))
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS)
```

```
    .build()
WorkManager.getInstance(context).enqueueUniqueWork(
    "offline_sync", ExistingWorkPolicy.REPLACE, syncRequest
)
```

19.3 Membuat Sale Offline

```
suspend fun createSale(input: CreateSaleInput): SaleResult {
    val idempotencyKey = UUID.randomUUID().toString()
    val inputWithDevice = input.copy(
        deviceCreatedAt = Instant.now().toString(),
        deviceId = Settings.Secure.getString(resolver, Settings.Secure.ANDROID_ID)
    )

    return if (isOnline()) {
        // Online: langsung kirim ke server
        try {
            api.createSale(tenantUuid, inputWithDevice, idempotencyKey)
        } catch (e: Exception) {
            // Gagal kirim → simpan offline
            saveOffline(idempotencyKey, inputWithDevice)
            SaleResult.SavedOffline
        }
    } else {
        // Offline: simpan di Room DB
        saveOffline(idempotencyKey, inputWithDevice)
        SaleResult.SavedOffline
    }
}
```

20. Checklist Header HTTP Android

Setiap request dari Android harus menyertakan header berikut via OkHttp Interceptor:

```
class AuthInterceptor(private val tokenManager: TokenManager) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val request = chain.request().newBuilder()
            .header("Authorization", "Bearer ${tokenManager.accessToken}")
            .header("Content-Type", "application/json")
            .header("Accept", "application/json")
            .build()
        return chain.proceed(request)
    }
}

// Untuk endpoint yang butuh idempotency (POST /sales):
.header("X-Idempotency-Key", idempotencyKey)
```

21. Alur Kerja Lengkap Aplikasi POS Android

21.1 Startup Flow

- **1. Login** — POST /auth/login → simpan access_token + refresh_token di EncryptedSharedPreferences
- **2. Pilih Tenant** — dari response login.user.tenants, user pilih tenant
- **3. Full Catalog Sync** — GET /sync/catalog → simpan ke Room DB (products, categories, tables, dll)
- **4. Cek Shift** — GET /shifts/current → ada shift open? Jika tidak, minta kasir buka shift
- **5. Tampilkan Layar Kasir** — produk dari Room DB, status dari server

21.2 Alur Transaksi Online

- **1. Kasir scan/pilih produk** → tambah ke cart lokal
- **2. Terapkan modifier** → modifier chip → note item
- **3. Hitung total sementara** → bisa preview diskon via GET /discount-rules/preview
- **4. Pilih metode bayar** → cash/card/qris. Untuk QRIS: cek internet dulu!
- **5. POST /sales** dengan X-Idempotency-Key → server hitung surcharge, pajak, diskon otomatis
- **6. Cetak struk** → GET /sales/:uuid/receipt/escpos → kirim bytes ke printer
- **7. Buka laci kas** (jika bayar tunai) → GET /devices/cash-drawer/open

21.3 Alur Transaksi Offline

- **1. Deteksi offline** via ConnectivityManager
- **2. Tampilkan indikator offline** — badge merah di status bar app
- **3. Buat transaksi** → simpan ke Room DB dengan status PENDING + idempotency key baru
- **4. ESC/POS tetap bisa cetak** — generate bytes lokal dari data lokal (tanpa call API)
- **5. QRIS tidak tersedia** — disable tombol QRIS saat offline
- **6. Background sync** — WorkManager dengan network constraint akan upload saat internet kembali
- **7. Notify kasir** — tampilkan notifikasi "X transaksi berhasil disinkronisasi"

21.4 Token Refresh Flow

```
// OkHttp Authenticator — otomatis handle 401
class TokenAuthenticator : Authenticator {
    override fun authenticate(route: Route?, response: Response): Request? {
        if (response.code == 401) {
            val newTokens = runBlocking {
                authApi.refresh(RefreshInput(tokenManager.refreshToken))
            }
            tokenManager.saveTokens(newTokens)
            return response.request.newBuilder()
                .header("Authorization", "Bearer ${newTokens.access_token}")
                .build()
```

```
        }
        return null
    }
}
```

22. Nilai Enum & Konstanta

22.1 Tipe Data Enum di Backend

Field	Nilai yang Valid
`order_type`	dine_in | takeaway | delivery
`payment_method`	cash | card | qris | transfer | other
`sale_status`	held | paid | void | served | cancelled
`kds_order_status`	new | cooking | ready | done
`kds_item_status`	pending | cooking | ready | done
`table_status`	available | occupied | reserved | maintenance
`shift_status`	open | closed
`printer_type`	receipt | kitchen | label
`conn_type`	bluetooth | network | usb
`discount_type`	nominal | percentage
`role`	staff | admin | owner

22.2 Format Mata Uang

Semua nilai uang di backend disimpan sebagai integer Rupiah (tidak ada sen). Response menggunakan tipe Decimal dari Rust yang muncul sebagai string atau number di JSON. Android harus parse ke Long atau BigDecimal.

```
// Contoh: harga 35000 dapat muncul sebagai:
"price": 35000     // number (lebih umum)
"price": "35000"   // string decimal

// Android: gunakan BigDecimal atau Long untuk menyimpan, Long untuk operasi
val price = response.price.toLong()  // selalu bilangan bulat
```

23. Checklist Implementasi Android

23.1 Phase 1: Core (MVP)

- Setup project: Kotlin + Compose + Hilt + Room + Retrofit
- Auth: Login screen, Google SSO, token storage (EncryptedSharedPreferences), refresh interceptor
- Tenant picker: setelah login, pilih tenant dari daftar
- Full catalog sync saat startup: GET /sync/catalog → Room DB
- Layar kasir: grid produk dari Room DB, search, filter kategori
- Barcode scanner: ML Kit → GET /products/barcode/:barcode
- Cart: tambah/edit/hapus item, modifier chip, price override
- Shift: buka/tutup shift, cek shift aktif saat startup
- Checkout: POST /sales dengan cash payment
- ESC/POS: cetak struk via Bluetooth printer

23.2 Phase 2: Offline & Advanced

- Offline detection: ConnectivityManager, badge status
- Offline sales: simpan ke Room DB saat tidak ada internet
- SyncWorker: WorkManager upload offline transactions
- QRIS payment: Xendit QR, polling status, disable saat offline
- Hold order: simpan order, bayar nanti via POST /sales/:uuid/pay
- Delta sync: updated_after untuk sync incremental
- Meja (Table Map): tampilkan status meja, assign ke order
- KDS (Kitchen Display): polling /kds, update status item
- Split payment: POST /sales dengan array payments

23.3 Phase 3: Manajemen (Admin)

- Order board: papan pesanan warung
- Laporan: summary, terlaris, per kasir
- Manajemen produk: list, search, stock adjustment
- Cash in & Expense: catat pemasukan dan pengeluaran
- Printer config: daftarkan printer ke server
- Voucher validation: input kode → preview diskon

© 2026 Rancak Platform — Dokumen ini dibuat otomatis dari source code backend.
Untuk pertanyaan teknis, silakan merujuk ke source code di /pos-backend/src/routes/